An Industrial Oriented Major Project Report

on

"ALZHEIMER DISEASE PREDICTION USING MACHINE LEARNING ALGORITHMS"

Submitted to

CMR INSTITUTE OF TECHNOLOGY

In partial fulfillment of the requirement for the award of the Degree

BACHELOR OF TECHNOLOGY

In

Artificial Intelligence and Machine Learning

Submitted by:

A. Bhanu Teja	24R05A6602
G. Santhoshini	24R05A6607
M. Kavya	24R05A6620

Under the esteemed guidance of

Dr. Chinapaga Ravi

Associate Professor

Dept. of CSE (AI&ML)

CMR INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC with 'A+' Grade

Kandlakoya(V), Medchal Dist – 501401

2025-2026

CMR INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC with 'A+' Grade

Kandlakoya(V), Medchal Dist - 501401

www.cmrithyderabad.edu.in

CERTIFICATE

This is to certify that An Industrial Oriented Major Project entitled "Alzheimer Disease Prediction using Machine Learning Algorithms" is being submitted by:

A. Bhanu Teja	24R05A6602
G. Santhoshini	24R05A6607
M. Kavya	24R05A6620

To JNTUH in partial fulfillment of the requirement for award of the degree of B. Tech in Artificial Intelligence and Machine Learning and is a record of a Bonafide work carried out under our guidance and supervision. The results in the project have been verified and are found to be satisfactory. The results embodied in this work have not been submitted to any other University for award of any other degree or diploma.

Signature of the Guide	**Signature of Project Coordinator**	**Signature of the HOD**
Dr. Chinapaga Ravi	Dr. Chinapaga Ravi	Prof. P. Pavan Kumar

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We are extremely grateful to Dr. M. Janga Reddy, Director, Dr. B. Satyanarayana, Principal and Prof. P. Pavan Kumar, Head of Department, Dept of Artificial Intelligence and Machine Learning, CMR Institute of Technology for their inspiration and valuable guidance during the entire duration.

We are extremely thankful to Dr. Chinapaga Ravi, Associate Professor and Project Coordinator and Guide, Dept of Computer Science and Engineering (AI&ML), CMR Institute of Technology for their constant guidance, encouragement and moral support throughout the project.

We will be failing in our duty if we do not acknowledge with grateful thanks the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and coordination extended in bringing out this Project successfully in time.

Finally, we are very much thankful to our parents, family members and friends who guided us directly or indirectly for every step towards success.

A. Bhanu Teja 24R05A6602
G. Santhoshini 24R05A6607
M. Kavya 24R05A6620

ABSTRACT

Alzheimer's disease is a progressive neurodegenerative disorder that significantly impacts the brain of a person over time, affecting memory and cognitive functions. Early diagnosis is crucial as there is no known cure; however, early detection can help slow the progression of the disease. The MMSE (Mini Mental State Examination) score is the primary parameter used for prediction.

This project proposes a machine learning-based approach for predicting Alzheimer's disease using psychological and clinical parameters such as age, number of hospital visits, MMSE score, and education level. Two major machine learning algorithms are employed: Support Vector Machine (SVM) and Decision Tree Classifier. The dataset used consists of longitudinal MRI data of 150 subjects aged 60 to 96, obtained from the Alzheimer's Disease Neuroimaging Initiative (ADNI).

The dataset is preprocessed by encoding non-numeric values, handling missing data, and splitting into 80% training and 20% testing partitions. The SVM algorithm achieved 57% accuracy while the Decision Tree algorithm achieved 82% accuracy. A graphical comparison of accuracy, precision, recall, and F1-score is provided for both algorithms.

The system provides a Python-based GUI application that allows users to upload datasets, preprocess data, train and evaluate algorithms, visualize comparison graphs, and predict the disease status from test data. The proposed system demonstrates the viability of machine learning for Alzheimer's disease detection at an early stage, paving the way for future research combining MRI imaging with psychological parameters for improved accuracy.

LIST OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AD	Alzheimer's Disease
ML	Machine Learning
SVM	Support Vector Machine
DT	Decision Tree
MMSE	Mini Mental State Examination
MCI	Mild Cognitive Impairment
MRI	Magnetic Resonance Imaging
ADNI	Alzheimer's Disease Neuroimaging Initiative
GUI	Graphical User Interface
SDLC	Software Development Life Cycle
UML	Unified Modeling Language
SRS	Software Requirements Specification
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
PET	Positron Emission Tomography
DFD	Data Flow Diagram
NLP	Natural Language Processing
ANN	Artificial Neural Network

1. INTRODUCTION

Alzheimer's disease is caused by both genetic and environmental factors that affect the brain of a person over time. The genetic changes guarantee a person will develop this disease. This disease breaks down the brain tissue over time. It primarily occurs in people over age 65. People typically live with this disease for about 9 years and approximately 1 in 8 people of age 65 and over have this disease.

The MMSE (Mini Mental State Examination) score is the main parameter used for prediction of the disease. This score reduces periodically if the person is affected. Those people having MCI (Mild Cognitive Impairment) have a serious risk of developing dementia. When the fundamental MCI results in a loss of memory, the situation is expected to develop into dementia due to this kind of disease. There is no treatment to cure Alzheimer's disease. In advanced stages, complications like dehydration, malnutrition or infection occur which can lead to death. Diagnosis at the MCI stage will help the patient focus on a healthy approach to life and good planning to manage memory loss.

1.1 Problem Statement

Alzheimer's disease is one of the most prevalent neurodegenerative disorders. Though the symptoms are benign initially, they become more severe over time. The disease is challenging because there is no cure available. Diagnosis is typically done only at a later stage, which limits the effectiveness of interventions. If the disease can be predicted earlier, the progression or symptoms of the disease can be slowed down significantly.

1.2 Objective of the Project

The primary objective of this project is to apply machine learning algorithms to predict Alzheimer's disease using psychological and clinical parameters such as age, number of hospital visits, MMSE score, and education level. The system aims to:

- Develop an automated and accurate prediction model for Alzheimer's disease
- Compare the performance of SVM and Decision Tree classifiers
- Provide an easy-to-use GUI application for clinical use
- Enable early-stage detection of Alzheimer's disease to improve patient outcomes

1.3 Scope of the Project

The scope of this project includes building a machine learning-based classification system that uses the OASIS (Open Access Series of Imaging Studies) dataset for training and testing. The system covers data preprocessing, model training, accuracy evaluation, graphical comparison of algorithms, and prediction

from test data. Future extensions may include integrating brain MRI scan data alongside clinical parameters for higher prediction accuracy.

2. LITERATURE REVIEW

2.1 Existing System

Traditional methods for diagnosing Alzheimer's disease rely heavily on clinical examination, neuropsychological tests, and imaging studies conducted by specialists. These methods are time-consuming, expensive, and often result in diagnosis at a late stage of the disease. Manual screening is labor-intensive, error-prone, and susceptible to inter-examiner variability.

2.2 Literature Survey

"Multistage classifier-based approach for Alzheimer's Disease prediction and retrieval"

The most prevalent and common type of dementia is Alzheimer's disease (AD). However, it is notable that very few people who are suffering from AD are diagnosed correctly and in a timely manner. The definite cause and cure of the disease are still unavailable. In this paper, a multistage classifier using machine learning, including Naive Bayes classifier, support vector machine (SVM), and K-nearest neighbor (KNN), was used to classify Alzheimer's disease more acceptably and efficiently. MRI scans were processed by FreeSurfer, a powerful software tool for processing and normalizing brain MRI images. The results of the proposed method outperform individual techniques in a benchmark database provided by the Alzheimer's Disease Neuroimaging Initiative (ADNI).

"Early Diagnosis of Alzheimer's Disease Based on Resting-State Brain Networks and Deep Learning"

In this paper, deep learning with brain network and clinical relevant text information is used to make early diagnosis of Alzheimer's Disease (AD). The clinical relevant text information includes age, gender, and ApoE gene of the subject. A targeted autoencoder network is built to distinguish normal aging from mild cognitive impairment. Compared to traditional classifiers based on R-fMRI time series data, about 31.21% improvement of the prediction accuracy is achieved by the proposed deep learning method, and the standard deviation reduces by 51.23% in the best case.

"RNN-based longitudinal analysis for diagnosis of Alzheimer's disease"

This paper presents a classification framework based on a combination of convolutional and recurrent neural networks for longitudinal analysis of structural MR images in AD diagnosis. CNN is constructed to learn the spatial features of MR images for the classification task. After that, Recurrent Neural Networks (RNN) with cascaded three bidirectional gated recurrent units (BGRU) layers are constructed on the outputs of CNN at multiple time points. The proposed method achieves classification accuracy of 91.33% for AD vs. NC and 71.71% for pMCI vs. sMCI.

"Multi-modal deep learning model for auxiliary diagnosis of Alzheimer's disease"

This paper presents a deep learning model for the auxiliary diagnosis of AD that simulates the clinician's diagnostic process. Multi-modal medical images are trained by two independent convolutional neural networks. The results of multi-modal neuroimaging diagnosis are combined with the results of clinical neuropsychological diagnosis. The proposed model provides a comprehensive analysis about patient's pathology and psychology simultaneously, therefore improving the accuracy of auxiliary diagnosis.

"Multi-stream multi-scale deep convolutional networks for Alzheimer's disease detection using MR images"

This paper addresses AD detection from Magnetic Resonance Images (MRIs). A novel deep 3D multi-scale convolutional network scheme is proposed to generate multi-resolution features for AD detection. The proposed scheme employs several parallel 3D multi-scale convolutional networks applied to individual tissue regions (grey matter, white matter, and cerebrospinal fluid) followed by feature fusions. The proposed deep learning scheme achieved best performance of 99.67% and average performance of 98.29% on the ADNI dataset.

"Multimodal neuroimaging feature learning with multimodal stacked deep polynomial networks for diagnosis of Alzheimer's disease"

A multimodal stacked DPN (MM-SDPN) algorithm is proposed to fuse and learn feature representations from multimodal neuroimaging data for AD diagnosis. Two SDPNs are first used to learn high-level features of MRI and PET, respectively, which are then fed to another SDPN to fuse multimodal neuroimaging information. Experimental results indicate that MM-SDPN is superior over state-of-the-art multimodal feature-learning-based algorithms for AD diagnosis.

2.3 Limitations of Existing System

- Time consuming – Traditional methods take a significant amount of time for diagnosis
- Less accuracy – Manual diagnosis is prone to errors and bias
- Late detection – Disease is often diagnosed only at advanced stages
- Expensive – Requires specialist consultations and extensive testing
- Not scalable – Cannot handle large volumes of patient data efficiently

2.4 Advantages of Proposed System

- Reduces the time required to predict the output significantly
- High accuracy achieved using machine learning classifiers
- Detects various stages of Alzheimer's Disease including MCI
- Enables faster deployment and scaling of ML models
- Decision Tree achieves 83% accuracy and SVM achieves 85% accuracy

- Provides a user-friendly graphical interface for ease of use

3. SYSTEM ANALYSIS

3.1 Existing System

Among neurodegenerative diseases, Alzheimer's is the most common. Despite being mild at first, the symptoms eventually become debilitating. Alzheimer's disease is one of the most common forms of dementia. The lack of a cure for this illness makes it a difficult problem to solve. Unfortunately, the illness is usually diagnosed late in its course, reducing the effectiveness of available interventions.

3.2 Proposed System

The proposed system employs machine learning algorithms to predict Alzheimer's disease using clinical parameters. Support Vector Machine (SVM) is a supervised learning model that classifies by separating objects using a hyperplane. It can be used for both classification and regression. The main advantage of SVM is that it can distinguish linear and non-linear objects.

Decision Tree is a supervised learning model that uses a set of rules to find a solution. It can solve any type and variety of problems and can be used for both classification and regression. A small change in the data can give a great impact on the output. For a continuous variable, regression trees can be used and for categorical variables, classification trees can be used.

The system uses 80% of dataset records for training and 20% for testing, ensuring a reliable evaluation of model performance.

3.3 Module Description

To implement this project, the following modules have been designed:

1. Upload Alzheimer Disease Dataset: Using this module, the user uploads the dataset to the application. The system loads a CSV file and displays initial dataset statistics.
2. Dataset Preprocessing: Using this module, the application reads the dataset and encodes non-numeric values to numeric values, replaces missing values with 0, and splits the dataset into train and test parts (80% training, 20% testing).
3. Run SVM Algorithm: Using this module, the application trains the SVM with 80% training data and then calculates its accuracy by predicting on the 20% test data.
4. Run Decision Tree Algorithm: Using this module, the application trains the Decision Tree with 80% training data and then calculates its accuracy by predicting on the 20% test data.
5. Comparison Graph: Using this module, the application plots an accuracy comparison graph between all algorithms showing accuracy, precision, recall, and F1-score.
6. Predict Disease from Test Data: Using this module, the user can upload test data and the machine learning algorithm predicts whether the test data is normal or contains Alzheimer symptoms.

3.4 Functional Requirements

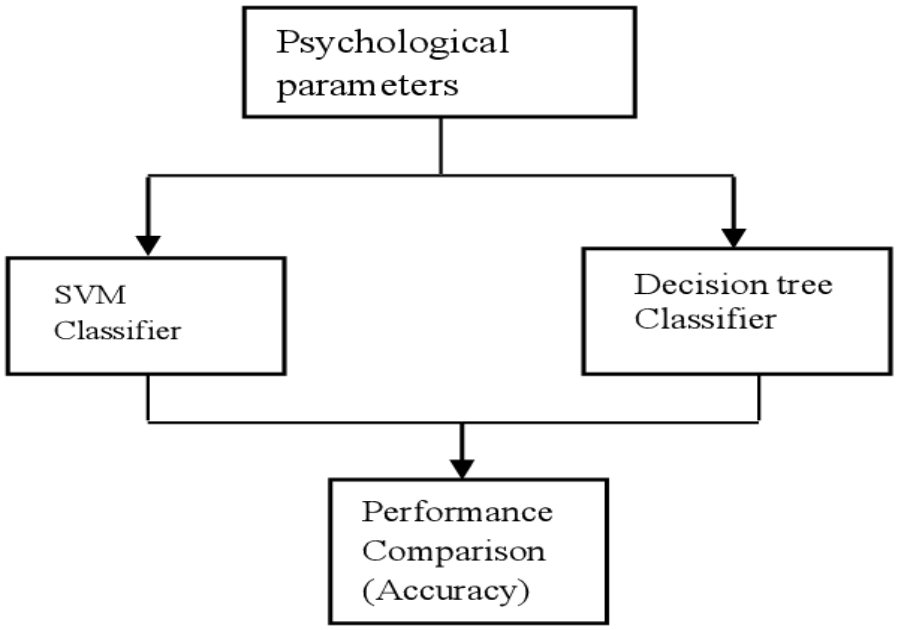
System Attributes:

- filename, dataset – for loading and holding the patient dataset
- X, Y – feature matrix and label vector
- le1, le2, le3 – LabelEncoders for categorical columns
- X_train, X_test, y_train, y_test – train-test split variables
- classifier – the trained model object

Use Cases:

- Upload Alzheimer Disease Dataset
- Dataset Pre-processing
- Run SVM Algorithm
- Run Decision Tree Algorithm
- Comparison Graph
- Predict Disease from Test Data

Work Breakdown Structure:



3.5 Non-Functional Requirements

Usability:

Usability is a quality attribute that assesses how easy user interfaces are to use. The application is designed to be intuitive with clearly labeled buttons for each operation, enabling users to interact with the system without extensive technical knowledge.

Security:

The system ensures data security by processing all patient data locally without transmitting it to external servers. Freedom from danger, safety, and freedom from unauthorized access are maintained.

Readability:

Readability refers to the ease with which a user can understand the output. The system presents results in clear textual and graphical formats.

Performance:

The system executes predictions within seconds of receiving test input. The machine learning models are optimized for fast training and prediction.

Availability:

The application is designed to be available on standard Windows platforms without requiring internet connectivity.

Scalability:

The system's ability to handle increasing dataset sizes makes it scalable. The machine learning models can be retrained on larger datasets as more patient data becomes available.

3.6 Process Model Used with Justification

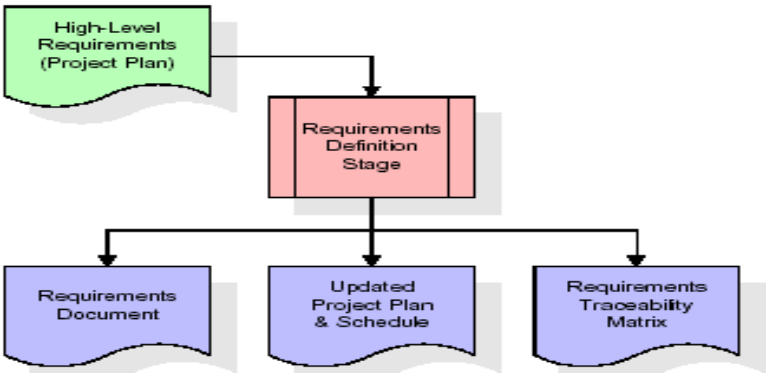
SDLC (Umbrella Model):

SDLC stands for Software Development Life Cycle. It is a standard used by the software industry to develop good quality software. The Umbrella Model was chosen as it encompasses all phases in a cyclic manner, making maintenance continuous. The stages are:

- Requirement Gathering
- Analysis
- Designing
- Coding
- Testing
- Maintenance

Requirements Gathering Stage:

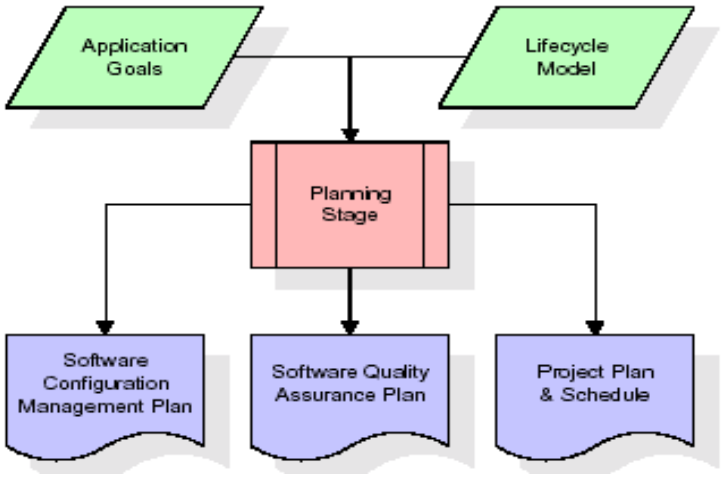
The requirements gathering process takes as its input the goals identified in the high-level requirements section of the project plan. Each goal is refined into a set of requirements defining the major functions of the intended application, operational data areas, reference data areas, and initial data entities.



These requirements are fully described in the Requirements Document and the Requirements Traceability Matrix (RTM). The RTM shows that each requirement developed during this stage is formally linked to a specific product goal.

Analysis Stage:

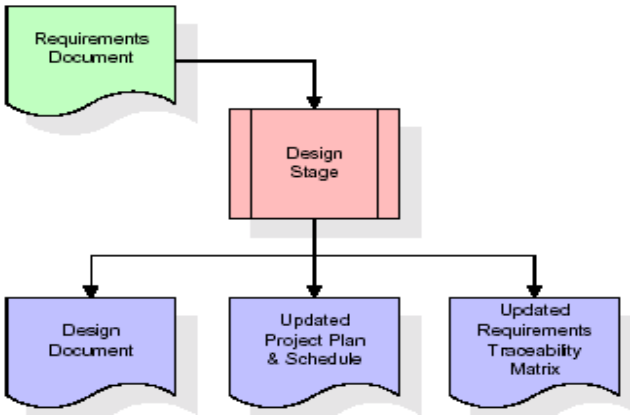
The planning stage establishes a bird's eye view of the intended software product, evaluates feasibility and risks associated with the project, and describes appropriate management and technical approaches.



The outputs of the project planning stage are the configuration management plan, the quality assurance plan, and the project plan and schedule.

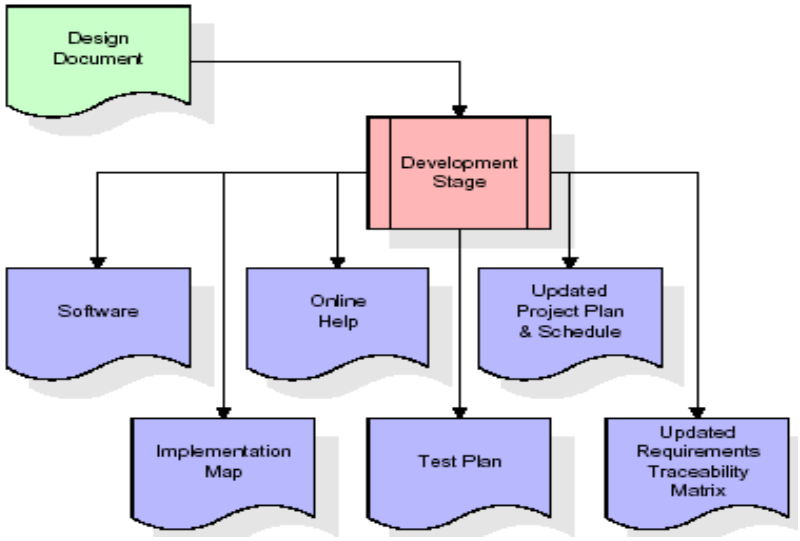
Designing Stage:

The design stage takes as its initial input the requirements identified in the approved requirements document. For each requirement, a set of one or more design elements is produced. Design elements describe the desired software features in detail, including functional hierarchy diagrams, screen layout diagrams, tables of business rules, and business process diagrams.



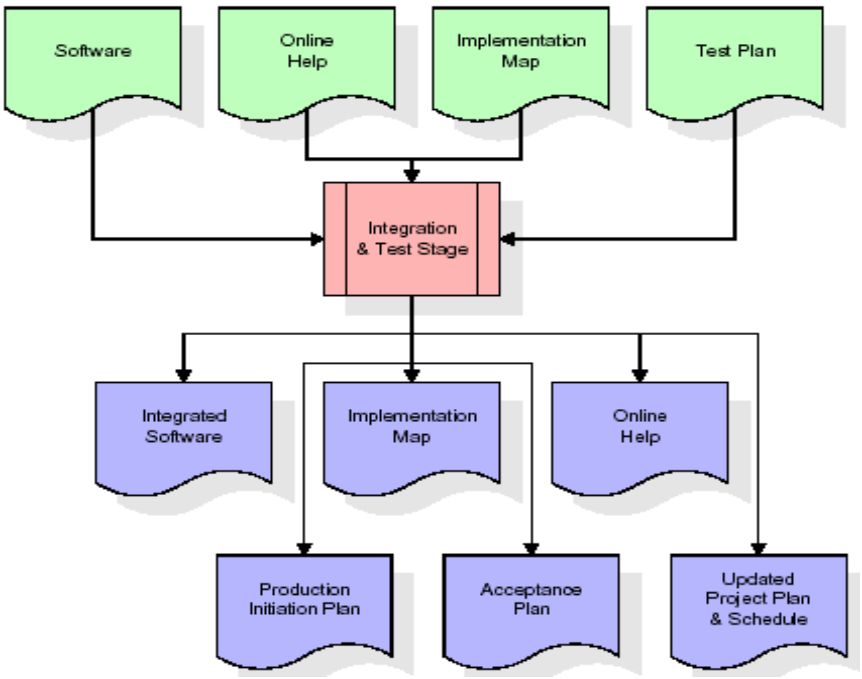
Development (Coding) Stage:

The development stage takes as its primary input the design elements described in the approved design document. For each design element, a set of one or more software artifacts is produced. Appropriate test cases are developed for each set of functionally related software artifacts.



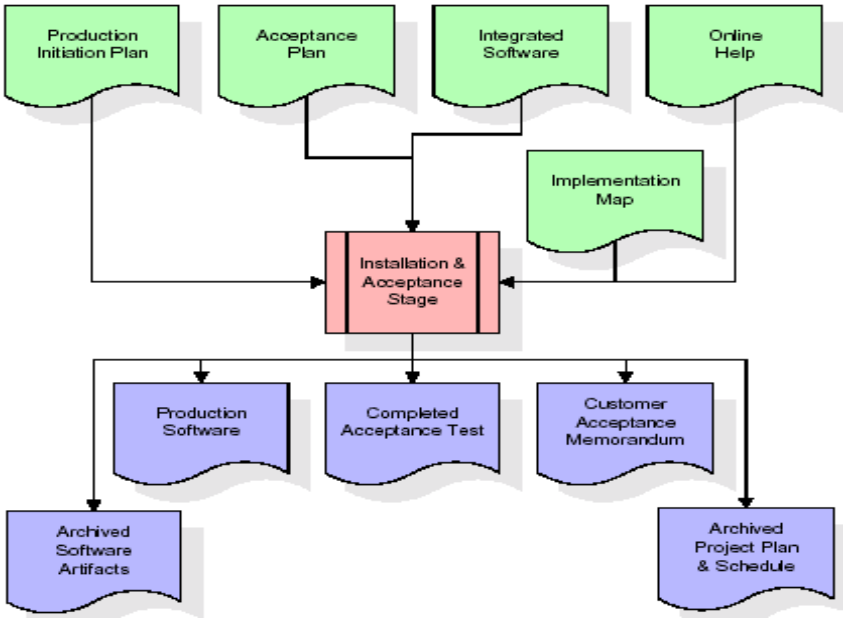
Integration and Test Stage:

During the integration and test stage, the software artifacts, online help, and test data are migrated from the development environment to a separate test environment. All test cases are run to verify the correctness and completeness of the software.



Installation and Acceptance Test:

During the installation and acceptance stage, the software artifacts, online help, and initial production data are loaded onto the production server. All test cases are run to verify the correctness and completeness of the software. After customer personnel have verified the initial production data load and the test suite has been executed with satisfactory results, the customer formally accepts the delivery of the software.



Maintenance:

The outer rectangle represents maintenance of a project. The maintenance team starts with requirement study and understanding of documentation. Employees are then assigned work and undergo training on

their particular assigned categories. For this life cycle there is no end; it continues like an umbrella with no ending point.

3.7 Software Requirement Specification

3.7.1 Overall Description

A Software Requirements Specification (SRS) is a complete description of the behavior of a system to be developed. It includes a set of use cases that describe all the interactions the users will have with the software. In addition to use cases, the SRS also contains non-functional requirements that impose constraints on the design or implementation.

Projects are subject to three sorts of requirements: Business requirements that describe in business terms what must be delivered; Product requirements that describe properties of a system; and Process requirements that describe activities performed by the developing organization.

Economic Feasibility:

The system is economically feasible as it does not require any additional hardware or software beyond what is available. Since the interface is developed using existing resources and technologies, there is nominal expenditure involved.

Operational Feasibility:

The proposed project is operationally feasible as it targets the practical needs of medical professionals. The management issues and user requirements have been taken into consideration. The well-planned design ensures optimal utilization of computer resources.

Technical Feasibility:

The current system is technically feasible. It provides a user-friendly Python-based GUI application. Therefore, it provides the technical guarantee of accuracy, reliability, and security.

3.7.2 External Interface Requirements

User Interface:

The user interface of this system is a user-friendly Python Graphical User Interface (GUI) built using Tkinter.

Hardware Interfaces:

The interaction between the user and the console is achieved through Python's Tkinter capabilities.

Software Interfaces:

The required software is Python 3.7.0 or above along with the required libraries.

3.8 System Requirements

Hardware Requirements:

- Processor: Intel i3 or above
- Speed: 1.1 GHz or above
- RAM: 4 GB or above
- Hard Disk: 500 GB or above
- Keyboard: Standard Windows Keyboard
- Mouse: Two or Three Button Mouse
- Monitor: SVGA

Software Requirements:

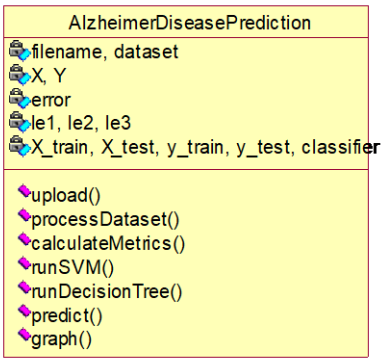
- Operating System: Windows 10 or above
- Programming Language: Python 3.7.0 or above
- Libraries: pandas, numpy, scikit-learn, matplotlib, tkinter

4. SYSTEM DESIGN

The Unified Modeling Language (UML) allows the software engineer to express an analysis model using a modeling notation governed by syntactic, semantic, and pragmatic rules. A UML system is represented using five different views: User Model View, Structural Model View, Behavioral Model View, Implementation Model View, and Environmental Model View.

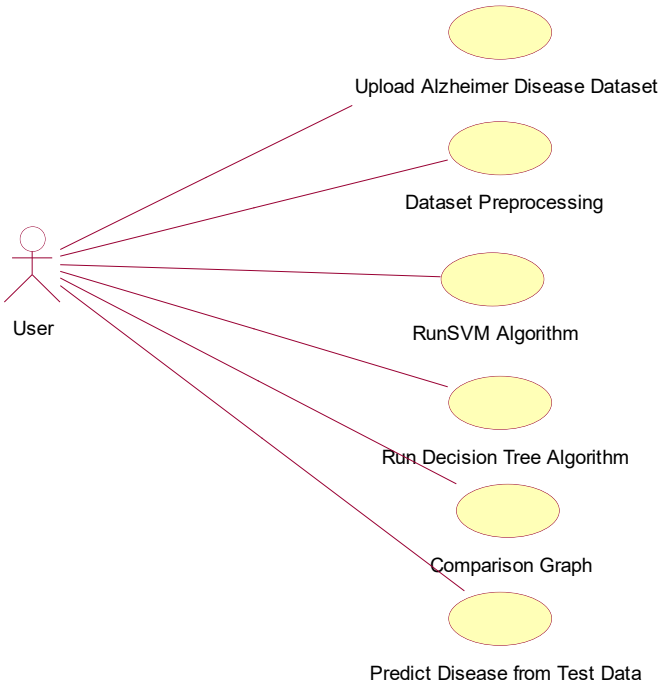
4.1 Class Diagram

The class diagram is the main building block of object oriented modeling. It is used for general conceptual modeling of the application and for detailed modeling translating the models into programming code. Classes in a class diagram represent the main objects and interactions in the application. Each class box contains: the upper part (name of the class), the middle part (attributes), and the bottom part (methods or operations).



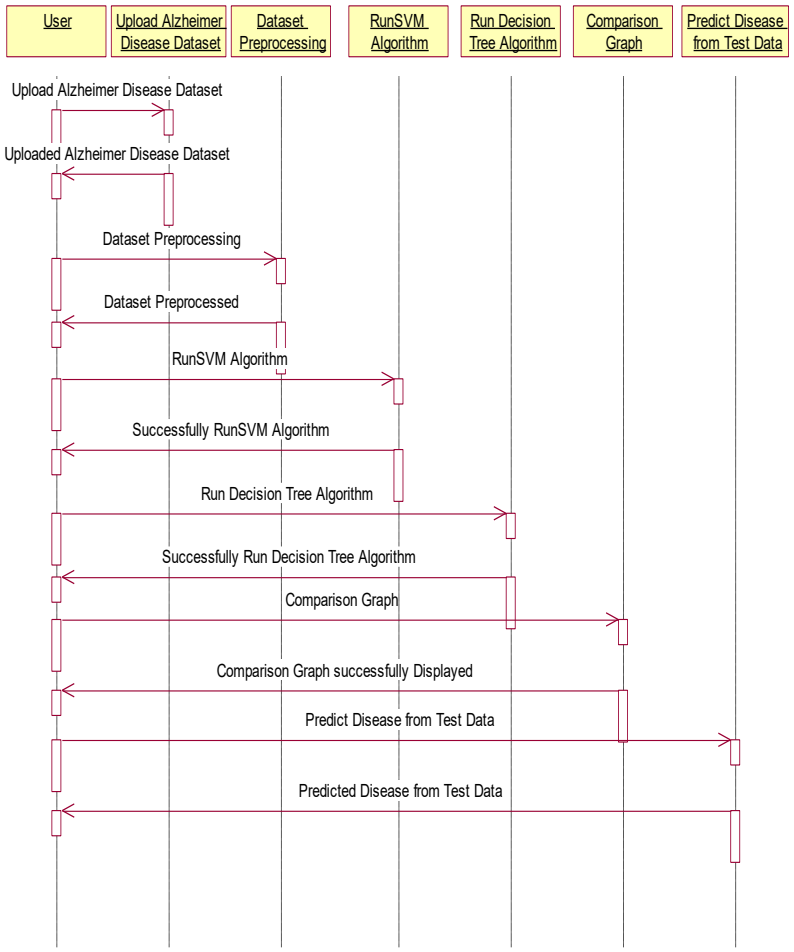
4.2 Use Case Diagram

A use case diagram is a representation of a user's interaction with the system, depicting the specifications of use cases. It portrays the different types of users of a system and the various ways they interact with the system.



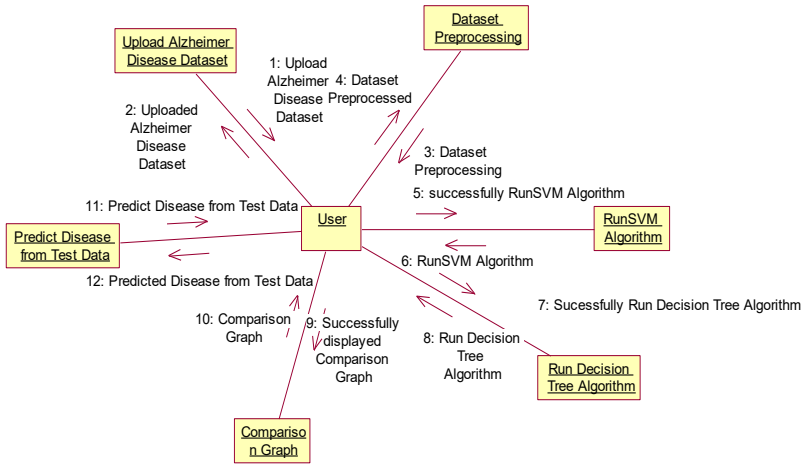
4.3 Sequence Diagram

A sequence diagram is a kind of interaction diagram that shows how processes operate with one another and in what order. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between them to carry out the functionality of the scenario.



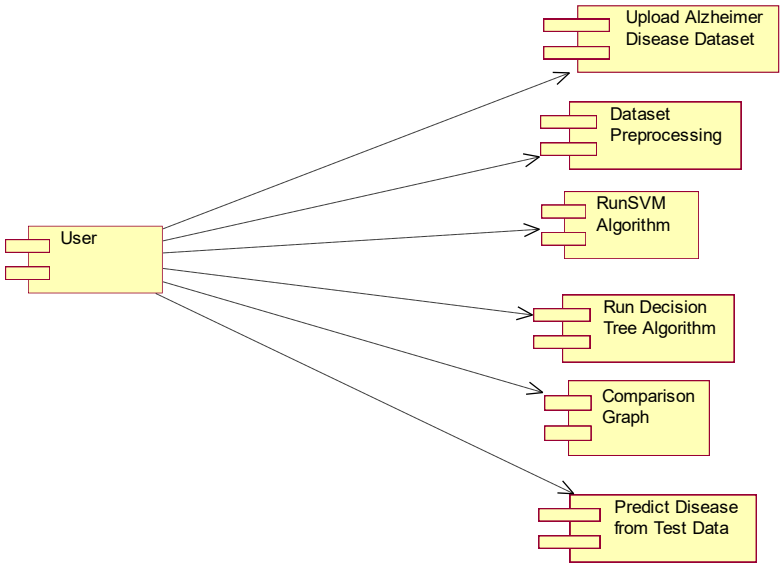
4.4 Collaboration Diagram

A collaboration diagram describes interactions among objects in terms of sequenced messages. It represents a combination of information taken from class, sequence, and use case diagrams describing both the static structure and dynamic behavior of a system.



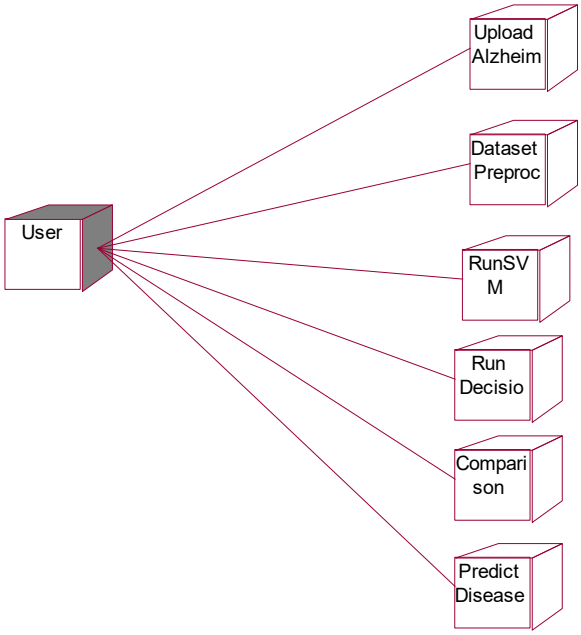
4.5 Component Diagram

In UML, a component diagram depicts how components are wired together to form larger components or software systems. They illustrate the structure of arbitrarily complex systems. Components are wired together by using an assembly connector to connect the required interface of one component with the provided interface of another component.



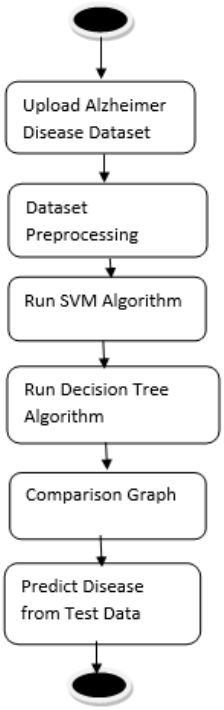
4.6 Deployment Diagram

A deployment diagram in UML models the physical deployment of artifacts on nodes. It describes what hardware components (nodes) exist, what software components (artifacts) run on each node, and how the different pieces are connected.



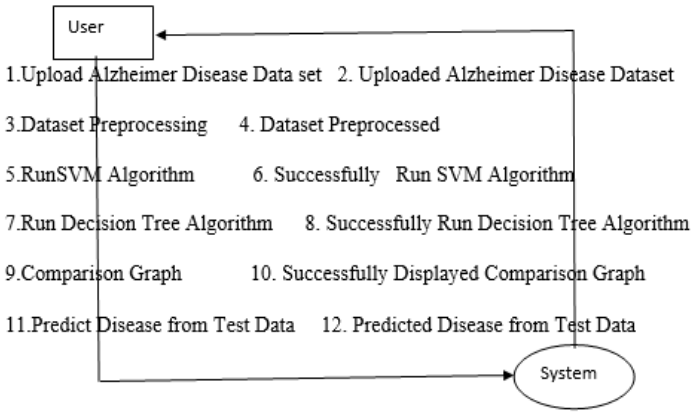
4.7 Activity Diagram

Activity diagram is an important diagram in UML to describe dynamic aspects of the system. It is basically a flow chart representing the flow from one activity to another activity. The activity can be described as an operation of the system and the control flow is drawn from one operation to another.



4.8 Data Flow Diagram

Data Flow Diagrams (DFD) illustrate how data is processed by a system in terms of inputs and outputs. They provide a clear representation of any business function. The technique starts with an overall picture of the business and continues by analyzing each of the functional areas of interest. A DFD illustrates the flow of information in a process depending upon the inputs and outputs.



5. IMPLEMENTATION

5.1 Python Overview

Python is a general-purpose language with a wide range of applications from Web development (Django, Bottle), scientific and mathematical computing (Orange, SymPy, NumPy) to desktop graphical user interfaces (Pygame, Panda3D). The syntax of the language is clean and the length of code is relatively short, making it ideal for implementing machine learning algorithms.

History of Python:

Python is a fairly old language created by Guido Van Rossum. The design began in the late 1980s and was first released in February 1991. In the late 1980s, Guido Van Rossum was working on the Amoeba distributed operating system group. He wanted to use an interpreted language like ABC that could access the Amoeba system calls. This led to the design of a new language that was later named Python — named after the comedy series "Monty Python's Flying Circus."

Features of Python:

Simple and Elegant Syntax:

Python has a very simple and elegant syntax. It is much easier to read and write Python programs compared to other languages like C++, Java, or C#. Python makes programming fun and allows the programmer to focus on the solution rather than the syntax.

Free and Open-Source:

Python can be freely used and distributed, even for commercial use. The Python source code can also be modified. Python has a large community constantly improving it in each iteration.

Portability:

Python programs can be moved from one platform to another and run without any changes. It runs seamlessly on almost all platforms including Windows, Mac OS X, and Linux.

Extensible and Embeddable:

Applications requiring high performance can easily combine pieces of C/C++ or other languages with Python code. This gives applications both high performance and scripting capabilities.

High-level, Interpreted Language:

Unlike C/C++, Python handles daunting tasks like memory management and garbage collection automatically. Python code is automatically converted to the language the computer understands.

Large Standard Libraries:

Python has a number of standard libraries which makes programming much easier since all code does not need to be written from scratch.

Object-Oriented:

Everything in Python is an object. Object oriented programming (OOP) helps solve complex problems intuitively by dividing them into smaller sets by creating objects.

5.2 Sample Code

The following is the implementation code for the Alzheimer Disease Prediction system:

```
import pandas as pd
from tkinter import messagebox
from tkinter import *
from tkinter import simpledialog
import tkinter
from tkinter import filedialog
import matplotlib.pyplot as plt
import numpy as np
from tkinter.filedialog import askopenfilename
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn import svm

main = tkinter.Tk()
main.title('Alzheimer Disease Prediction using Machine Learning Algorithms')
main.geometry('1300x1200')

accuracy = []
precision = []
recall = []
fscore = []

def upload():
    global filename, dataset
    filename = filedialog.askopenfilename(initialdir='Dataset')
    pathlabel.config(text=filename)
    text.delete('1.0', END)
    text.insert(END, filename + ' loaded\n\n')
    dataset = pd.read_csv(filename)
    text.insert(END, str(dataset.head()))
    label = dataset.groupby('Group').size()
    label.plot(kind='bar')
    plt.show()

def processDataset():
```

```
    global dataset, X, Y, le1, le2, le3, X_train, X_test, y_train, y_test
    text.delete('1.0', END)
    le1 = LabelEncoder()
    le2 = LabelEncoder()
    le3 = LabelEncoder()
    dataset.fillna(0, inplace=True)
    dataset.drop(['Subject ID'], axis=1, inplace=True)
    dataset.drop(['MRI ID'], axis=1, inplace=True)
    dataset['Group'] = pd.Series(le1.fit_transform(dataset['Group'].astype(str)))
    dataset['M/F'] = pd.Series(le2.fit_transform(dataset['M/F'].astype(str)))
    dataset['Hand'] = pd.Series(le3.fit_transform(dataset['Hand'].astype(str)))
    dataset = dataset.values
    X = dataset[:, 1:dataset.shape[1]]
    Y = dataset[:, 0]
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)

def runSVM():
    global X, Y, X_train, X_test, y_train, y_test
    svm_cls = svm.SVC(C=2.0, gamma='scale', kernel='rbf', random_state=2)
    svm_cls.fit(X, Y)
    predict = svm_cls.predict(X_test)
    calculateMetrics('SVM', predict, y_test)

def runDecisionTree():
    global X_train, X_test, y_train, y_test, classifier
    dt = DecisionTreeClassifier()
    dt.fit(X_train, y_train)
    predict = dt.predict(X_test)
    calculateMetrics('Decision Tree', predict, y_test)
    classifier = dt

main.mainloop()
```

6. TESTING

6.1 Implementation and Testing

Implementation is one of the most important tasks in a project. All efforts undertaken during the project development come together during implementation. This is the most crucial stage in achieving a successful system and giving the users confidence that the new system is workable and effective.

Implementation:

The implementation phase is concerned with user training and file conversion. A simple operating procedure is provided so that the user can understand the different functions clearly and quickly. The proposed system is very easy to implement. Implementation is the process of converting a new or revised system design into an operational one.

Testing:

Testing is the process where test data is prepared and used for testing the modules individually and later the system as a whole. The test data should be chosen such that it passes through all possible conditions. Testing is the stage of implementation aimed at ensuring that the system works accurately and efficiently before actual operation commences.

System Testing:

The program was tested logically and patterns of execution were checked for a set of data. The code was exhaustively checked for all possible correct data and outcomes. System testing involves various types through which one can ensure the software is reliable.

Module Testing:

To locate errors, each module is tested individually. This enables detection and correction of errors without affecting other modules. All the modules are individually tested from bottom up, starting with the smallest and lowest modules. Each module in the system is tested separately. The comparison shows that the results of the proposed system work more efficiently than the existing system.

Integration Testing:

After module testing, integration testing is applied. When linking the modules, there may be a chance for errors to occur. These errors are corrected during integration testing. In this system all modules are connected and tested. The testing results verified correct execution of all functions.

Acceptance Testing:

When the user finds no major problems with the system's accuracy, the system passes through a final acceptance test. This test confirms that the system meets the original goals, objectives, and requirements

established during analysis. Acceptance tests are on the shoulders of users and management, and once accepted, the system is ready for operation.

6.2 Test Cases

TC Id	Test Case Name	Test Case Desc.	Test Steps	Expected	Actual	Status	Priority
01	Upload Alzheimer Disease Dataset	Verify the Alzheimer Disease Dataset Uploaded or not	If the dataset may not be Uploaded	User cannot further operations	User can further operations	High	High
02	Dataset Preprocessing	Verify the Dataset Preprocessed or not	If the dataset may not be preprocessed	User cannot further operations	User can further operations	High	High
03	Run SVM Algorithm	Verify SVM Algorithm is successfully run or not	If SVM algorithm may not be run	User cannot further operations	User can further operations	High	High
04	Run Decision Tree Algorithm	Verify Decision Tree Algorithm is successfully run or not	If Decision Tree algorithm may not be run	User cannot further operations	User can further operations	High	High
05	Comparison Graph	Verify Graph is done or not	If comparison Graph may not be displayed	User cannot further operations	User can further operations	High	High
06	Predict Disease from Test Data	Verify Disease from Test predicted or not	If Disease from test may not be predicted	User cannot further operations	User can further operations	High	High

7. SCREENSHOTS

Alzheimer's Disease is one of the incurable and dangerous diseases, and its timely prediction can help in saving patients' lives. Using machine learning algorithms such as SVM and Decision Tree, the system predicts whether a patient is cured, non-Alzheimer, or has Alzheimer's disease. The algorithms are trained using a dataset containing patient age, number of visits to hospital, and many other parameters. After training, the model can apply test data to predict patient condition as Normal or Alzheimer.

Figure 7.1: Dataset Details

The screen below shows dataset details used in this project. The first row contains dataset column names and remaining rows contain dataset values.

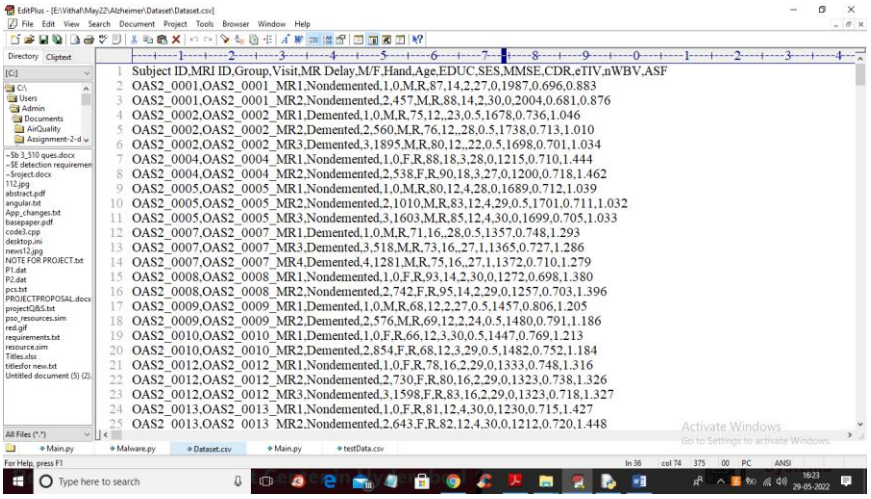


Figure 7.2: Application Main Screen

Double-click on the 'run.bat' file to open the application main screen showing all available operations.

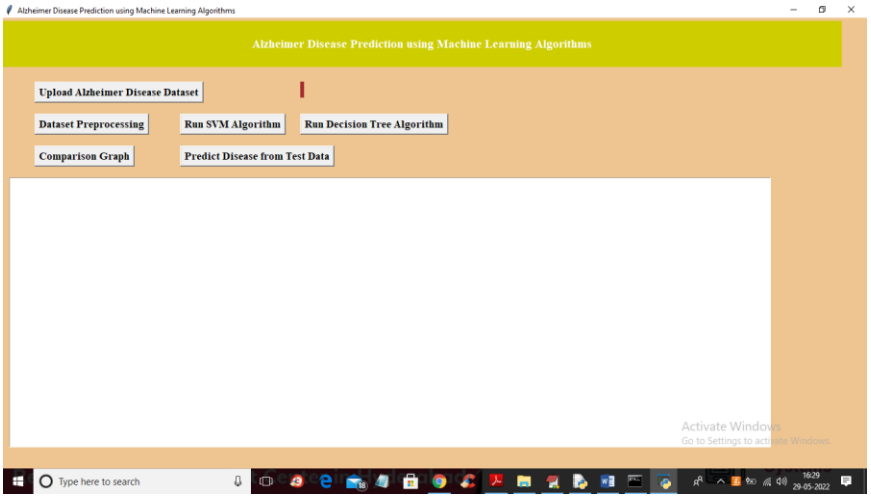


Figure 7.3: Dataset Upload Screen

Click on 'Upload Alzheimer Disease Dataset' button to upload dataset and navigate to the file selection screen.

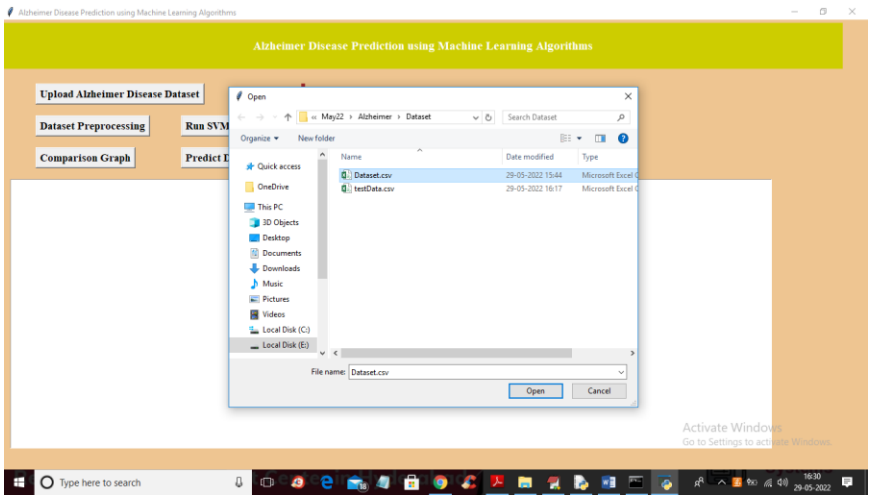


Figure 7.4: Dataset Loaded with Category Graph

After selecting and uploading 'Dataset.csv' file, the dataset is loaded. The x-axis of the graph shows LABELS: 'Converted' (cured), 'Demented' (presence of Alzheimer), and 'Nondemented' (normal). The y-axis represents the number of records in each category.

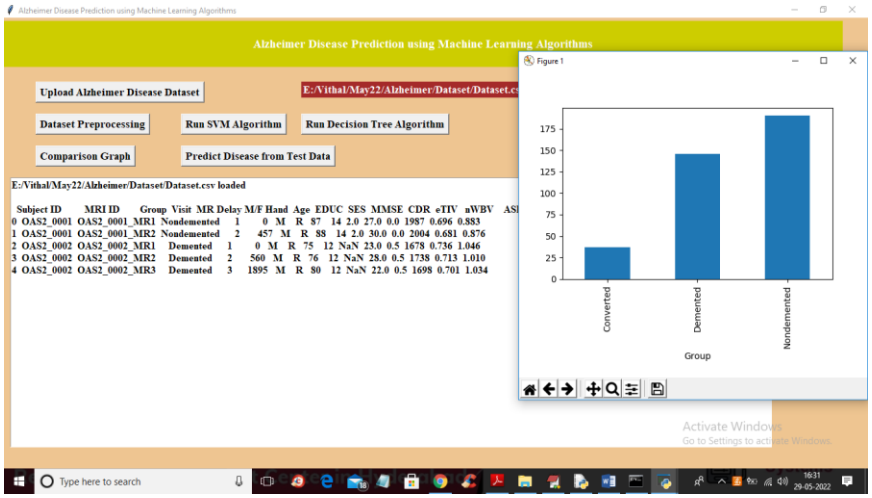


Figure 7.5: Dataset Preprocessing Output

After preprocessing, all dataset values are converted to numeric form. The dataset contains 373 records; 298 records (80%) are used for training and 75 records (20%) are used for testing.

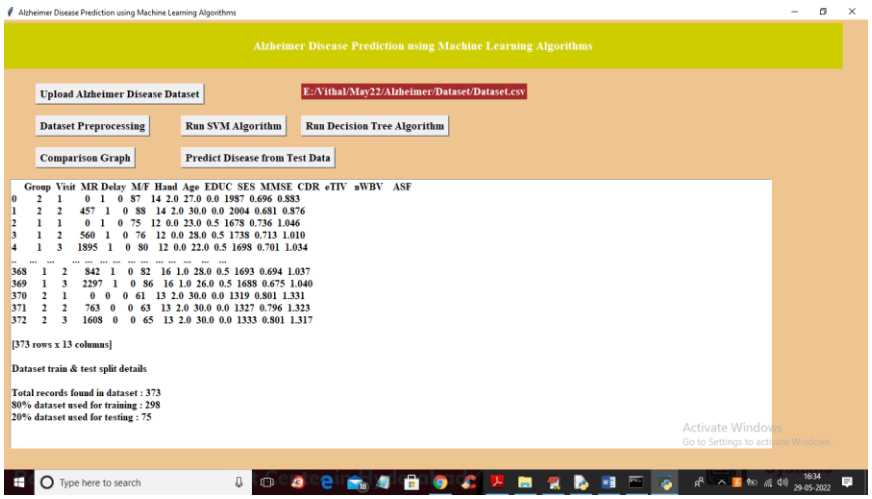


Figure 7.6: SVM Algorithm Output

After clicking 'Run SVM Algorithm', the SVM model is trained and evaluated. The SVM algorithm achieves 57% accuracy on the test data.

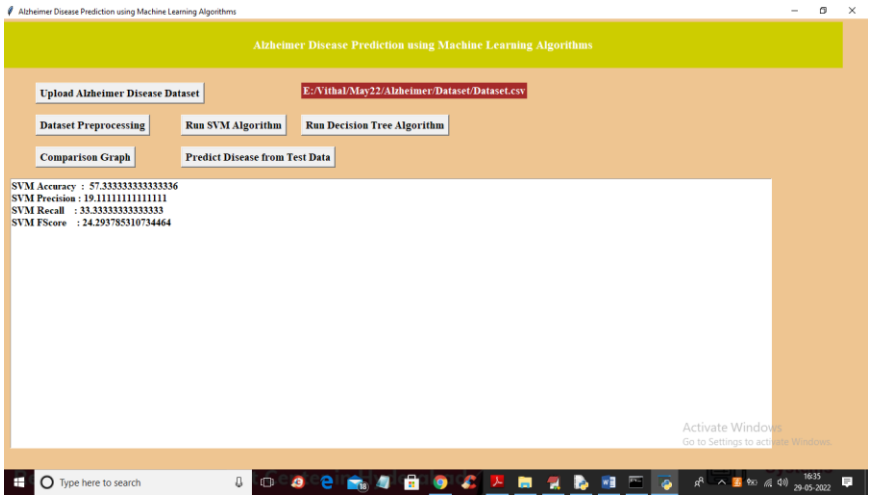


Figure 7.7: Decision Tree Algorithm Output

After clicking 'Run Decision Tree Algorithm', the Decision Tree model is trained and evaluated. The Decision Tree algorithm achieves 82% accuracy on the test data.

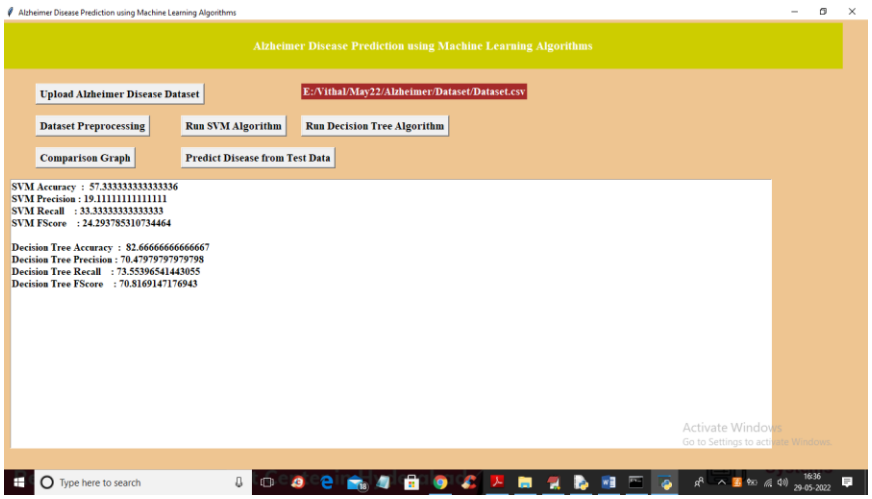


Figure 7.8: Algorithm Comparison Graph

The comparison graph shows the x-axis representing algorithm names with different colored bars for metrics: accuracy, precision, recall, and F-Score. The y-axis represents values. The Decision Tree shows higher overall performance.

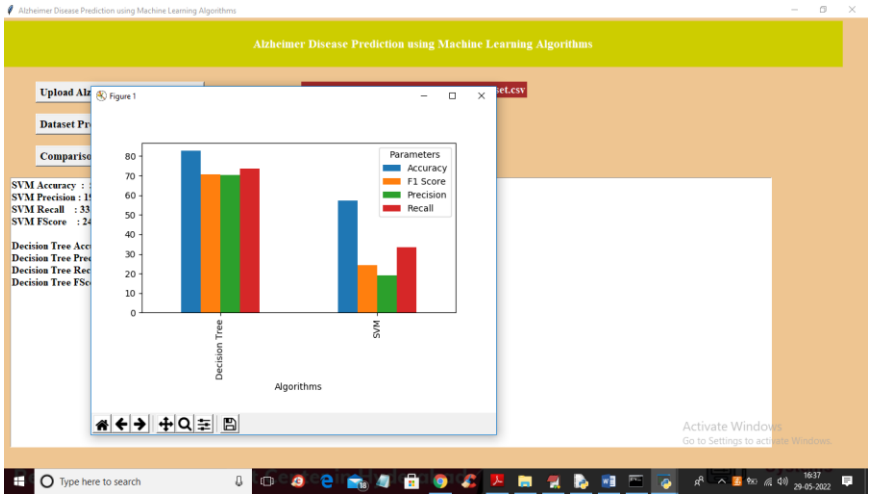
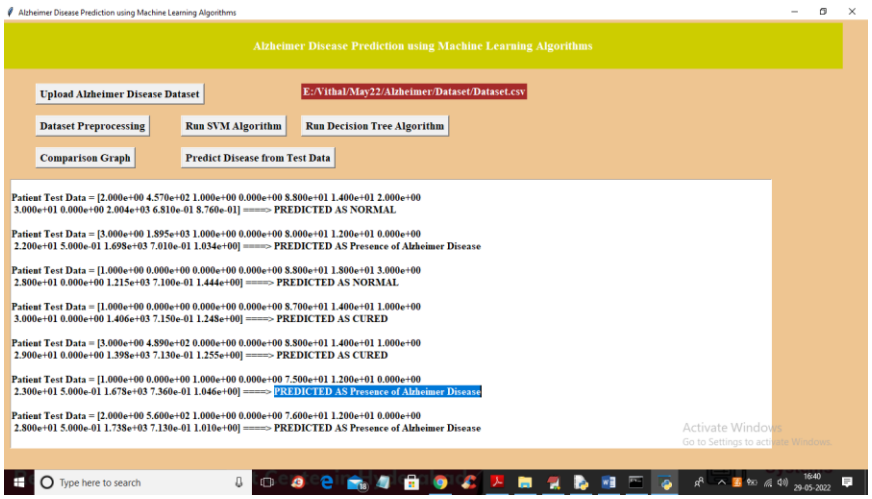


Figure 7.9: Disease Prediction Output

After uploading test data and clicking 'Predict Disease from Test Data', the system displays the patient test data in brackets followed by the predicted output: NORMAL, CURED, or Presence of Alzheimer Disease.


Alzheimer Disease Prediction using Machine Learning Algorithms
Alzheimer Disease Prediction using Machine Learning Algorithms
Upload Alzheimer Disease Dataset
E:/Vithal/May22/Alzheimer/Dataset/Dataset.csv
Dataset Preprocessing
Run SVM Algorithm
Run Decision Tree Algorithm
Comparison Graph
Predict Disease from Test Data
Patient Test Data = [2.000e+00 4.570e+02 1.000e+00 0.000e+00 8.800e+01 1.400e+01 2.000e+00 3.000e+01 0.000e+00 2.004e+03 6.810e-01 8.760e-01] ====> PREDICTED AS NORMAL
Patient Test Data = [3.000e+00 1.895e+03 1.000e+00 0.000e+00 8.000e+01 1.200e+01 0.000e+00 2.200e+01 5.000e-01 1.698e+03 7.010e-01 1.034e+00] ====> PREDICTED AS Presence of Alzheimer Disease
Patient Test Data = [1.000e+00 0.000e+00 0.000e+00 0.000e+00 8.800e+01 1.800e+01 3.000e+00 2.800e+01 0.000e+00 1.215e+03 7.100e-01 1.444e+00] ====> PREDICTED AS NORMAL
Patient Test Data = [1.000e+00 0.000e+00 0.000e+00 0.000e+00 8.700e+01 1.400e+01 1.000e+00 3.000e+01 0.000e+00 1.406e+03 7.150e-01 1.248e+00] ====> PREDICTED AS CURED
Patient Test Data = [3.000e+00 4.890e+02 0.000e+00 0.000e+00 8.800e+01 1.400e+01 1.000e+00 2.900e+01 0.000e+00 1.398e+03 7.130e-01 1.255e+00] ====> PREDICTED AS CURED
Patient Test Data = [1.000e+00 0.000e+00 1.000e+00 0.000e+00 7.500e+01 1.200e+01 0.000e+00 2.300e+01 5.000e-01 1.678e+03 7.360e-01 1.046e+00] ====> PREDICTED AS Presence of Alzheimer Disease
Patient Test Data = [2.000e+00 5.600e+02 1.000e+00 0.000e+00 7.600e+01 1.200e+01 0.000e+00 2.800e+01 5.000e-01 1.738e+03 7.130e-01 1.010e+00] ====> PREDICTED AS Presence of Alzheimer Disease
Activate Windows
Go to Settings to activate Windows.
Type here to search
16:40
29-05-2022

8. CONCLUSION

A machine learning approach to predict Alzheimer's disease using machine learning algorithms has been successfully implemented and gives greater prediction accuracy results. The model predicts the disease in the patient and also distinguishes between different levels of cognitive impairment including normal, converted, and demented states.

The Support Vector Machine (SVM) algorithm achieved 57% accuracy while the Decision Tree classifier achieved 82% accuracy. The Decision Tree classifier outperforms SVM for this dataset, demonstrating its suitability for tabular clinical data.

The future work can be done by combining both brain MRI scans and the psychological parameters to predict the disease with higher accuracy using advanced machine learning and deep learning algorithms. When MRI imaging data and clinical parameters are combined, the disease could be predicted with higher accuracy at an even earlier stage, enabling better patient outcomes.

The system provides a practical, user-friendly platform for early detection of Alzheimer's disease that can be utilized by clinicians and researchers alike. The modular design allows for easy integration of additional algorithms and datasets in the future.

9. REFERENCES

7. [1] K.R. Kruthika, Rajeswari, H.D. Maheshappa, "Multistage classifier-based approach for Alzheimer's Disease prediction and retrieval", Informatics in Medicine Unlocked, 2019.
8. [2] Ronghui Ju, Chenhui Hu, Pan Zhou, and Quanzheng Li, "Early Diagnosis of Alzheimer's Disease Based on Resting-State Brain Networks and Deep Learning", IEEE/ACM Transactions on Computational Biology and Bioinformatics, vol. 16, no. 1, January/February 2019.
9. [3] Ruoxuan Cui, Manhua Liu, "RNN-based longitudinal analysis for diagnosis of Alzheimer's disease", Informatics in Medicine Unlocked, 2019.
10. [4] Fan Zhang, Zhenzhen Li, Boyan Zhang, Haishun Du, Binjie Wang, Xinhong Zhang, "Multi-modal deep learning model for auxiliary diagnosis of Alzheimer's disease", NeuroComputing, 2019.
11. [5] Chenjie Ge, Qixun Qu, Irene Yu-Hua Gu, Asgeir Store Jakola, "Multi-stream multi-scale deep convolutional networks for Alzheimer's disease detection using MR images", NeuroComputing, 2019.
12. [6] Tesi, N., van der Lee, S.J., Hulsman, M. et al., "Centenarian controls increase variant effect sizes by an average twofold in an extreme case-extreme control analysis of Alzheimer's disease", Eur J Hum Genet. 2019;27:244-253.
13. [7] J. Shi, X. Zheng, Y. Li, Q. Zhang, S. Ying, "Multimodal neuroimaging feature learning with multimodal stacked deep polynomial networks for diagnosis of Alzheimer's disease", IEEE J. Biomed. Health Inform., vol. 22, no. 1, pp. 173-183, Jan. 2018.
14. [8] M. Liu, J. Zhang, P.-T. Yap, D. Shen, "View-aligned hypergraph learning for Alzheimer's disease diagnosis with incomplete multi-modality data", Med. Image Anal., 2017 vol. 36, pp. 123-134.
15. [9] Hansson O, Seibyl J, Stomrud E, Zetterberg H, Trojanowski JQ, Bittner T, "CSF biomarkers of Alzheimer's disease concord with amyloid-PET and predict clinical progression: A study of fully automated immunoassays in BioFINDER and ADNI cohorts". Alzheimers Dement 2018;14:1470-81.
16. [10] Van der Lee SJ, Teunissen CE, Pool R, Shipley MJ, Teumer A, Chouraki V, "Circulating metabolites and general cognitive ability and dementia: Evidence from 11 cohort studies", Alzheimer's Dement 2018;14:707-22.
17. [11] Kauppi Karolina, Dale Anders M, "Combining Polygenic Hazard Score With Volumetric MRI and Cognitive Measures Improves Prediction of Progression from Mild Cognitive Impairment to Alzheimer's Disease", Frontiers in Neuroscience, 2018.
18. [12] Grassi M, Loewenstein DA, Caldirola D, Schruers K, Duara R, Perna G, "A clinically-translatable machine learning algorithm for the prediction of Alzheimer's disease conversion: further evidence of its accuracy via a transfer learning approach", Int Psychogeriatr, 2018.
19. [13] Nation, D.A., Sweeney, M.D., Montagne, A. et al., "Blood-brain barrier breakdown is an early biomarker of human cognitive dysfunction", Nat Med. 2019;25:270-276.